



AutoAIM Studio

Auto Artificial Intelligence for Materials

User Manual

Version 1.4.0

Codename: "Full Control"

June 27, 2026

Contact: veradelagarza@quimica.unam.mx

DOI: [10.5281/zenodo.20947495](https://doi.org/10.5281/zenodo.20947495)

Table of Contents

1. Introduction

2. Getting Started

2.1 System Requirements

2.2 Installation

2.3 First Launch

3. Data Tab

3.1 Loading Data

3.2 Crystal Structure Support

3.3 Featurization

3.4 Target Selection

3.5 Train/Test Split

4. Training Tab

4.1 Algorithm Selection

4.2 Cross-Validation

4.3 Train with Custom Parameters

4.4 Understanding Results

5. Optimization Tab

5.1 Bayesian Optimization

5.2 Select Parameters to Optimize

5.3 Optimization Results

5.4 Train with Best Parameters

6. Neural Network Tab

6.1 Architecture Design

6.2 Training Configuration

6.3 NN Optimization

6.4 Training Results

7. Ensemble Tab

7.1 Creating Ensembles

7.2 Manual Ensemble Weights

7.3 Weight Optimization

8. Results Tab

9. Explainability Tab

10. Predict Tab

11. Model Bundles

12. Troubleshooting

13. Version History

14. Citation

1. Introduction

AutoAIM Studio is a standalone desktop application for automated machine learning in materials science and beyond. It provides a complete no-code AutoML pipeline with custom compositional feature engineering, crystal structure analysis, neural network design, hyperparameter optimization, manual parameter configuration, parameter selection for optimization, and standalone model deployment.

This manual covers all features of AutoAIM Studio v1.4.0 (codename "Full Control"), released on June 27, 2026. Whether you are predicting material properties from chemical formulas, analyzing crystal structures, or designing neural networks for complex regression tasks, this guide will walk you through every step.

Materials Science Context: Materials informatics combines materials science with machine learning to accelerate the discovery and optimization of new materials. AutoAIM Studio bridges the gap between domain expertise and ML implementation, allowing researchers to focus on science rather than code.

Pro Tip: If you are new to AutoAIM Studio, start with the Quick Start Guide in Section 2, then follow the tab-by-tab walkthrough in Sections 3-10.

Note: For support or questions, contact us at veradelagarza@quimica.unam.mx

2. Getting Started

2.1 System Requirements

Component	Minimum	Recommended
OS	Windows 10/11 (64-bit)	Windows 11 (64-bit)
CPU	4 cores	8+ cores
RAM	8 GB	16 GB+
Disk	2 GB free	5 GB free
GPU	Not required	Optional (CUDA for NN acceleration)
Display	1280x720	1920x1080 or higher

Note: The PyInstaller-compiled executable is CPU-only. For GPU support with PyTorch CUDA acceleration, run AutoAIM Studio from source with a CUDA-capable Python environment.

2.2 Installation

Windows (Pre-built Executable):

1. Download AutoAIM-Studio-v1.4.0-Windows.zip from the releases page
2. Extract to your preferred location
3. Run MaterialsAutoML.exe

No Python installation is required. All dependencies are bundled in the executable.

From Source:

```
git clone https://github.com/yourusername/materials_automl.git
cd materials_automl
pip install -r requirements.txt
python main_executable.py
```

2.3 First Launch

When you launch AutoAIM Studio for the first time, the application initializes its workspace and checks for required dependencies. The main window opens with the Data tab active, showing an empty dataset ready for loading.

Pro Tip: On first run with crystal structure files, pymatgen data is downloaded automatically. Ensure you have an active internet connection for this one-time setup.

3. Data Tab

The Data tab is your starting point for any AutoAIM Studio workflow. Here you load datasets, apply featurization, select your target variable, and configure train/test splitting. AutoAIM Studio supports both CSV files with chemical formulas and crystal structure files (CIF, POSCAR, XYZ).

3.1 Loading Data

CSV Files:

- Click "Load CSV" to import a dataset. The file must contain a target column (the property you want to predict) and at least one feature column or a formula/composition column for featurization.
- Supported formats: .csv, .xlsx, .xls, .parquet, .json
- String columns (composition, file paths, IDs) are automatically excluded from features

Crystal Structure Files:

- Click "Load Directory" to import all structure files from a folder
- Click "Load Files" for individual file selection

- Supported formats: CIF, POSCAR, CONTCAR, VASP, XYZ

Format	Extension	Description
CIF	.cif	Crystallographic Information File
POSCAR	.poscar	VASP position file
CONTCAR	.contcar	VASP continuation file
XYZ	.xyz	Cartesian coordinates
VASP	.vasp	Generic VASP format

Materials Science Context: Crystal structure files contain 3D atomic positions and lattice information. When loaded, AutoAIM automatically extracts approximately 154 numeric descriptors per structure, including lattice parameters, composition statistics, Magpie descriptors, bond analysis, and radial distribution function peaks. These descriptors are pure numeric vectors that feed directly into the training pipeline.

3.2 Crystal Structure Support

AutoAIM Studio v1.2.0+ provides comprehensive crystal structure support. When you load structure files, the application automatically extracts numeric features through the following pipeline:

Category	Count	Examples
Lattice	~10	a, b, c, alpha, beta, gamma, volume, symmetry flags
Composition	~13	avg atomic number, electronegativity, radii, ionization energy
Structural	~3	volume_per_atom, packing_fraction, spacegroup_number
Magpie	106	mean/std/min/max/median/p25/p75 for 13 elemental properties
Advanced	~15	bond lengths, coordination numbers, RDF peaks, lattice anisotropy
Derived	~2	density, total_electrons

Note: The manual featurizer ("Apply Featurizer" button) is designed for CSV files with a formula or composition column. Crystal structure datasets are already fully featurized upon loading.

3.3 Featurization

For CSV datasets containing chemical formulas or composition strings, AutoAIM Studio provides custom compositional featurization that generates 106 Magpie descriptors without requiring MatMiner.

To apply featurization:

4. Load your CSV file (ensure it has a 'formula' or 'composition' column)
5. Click "Apply Featurizer"
6. The application generates 106 descriptors including:
 - Elemental properties (40 descriptors): atomic number, row, column, ...
 - Statistical moments (24 descriptors): mean, std, min, max, ...
 - Composition ratios (12 descriptors)
 - Electronic structure (18 descriptors)
 - Thermodynamic estimates (12 descriptors)

Pro Tip: For crystal structure datasets, clicking "Apply Featurizer" shows an informative dialog explaining that ~154 features are already present, instead of a confusing error message.

3.4 Target Selection

After loading data, you must select the target column (the property you want to predict). When loading a target CSV for crystal structures, a dialog allows you to select which numeric column is the property to predict. All other columns are automatically discarded.

Important: Only numeric columns can be used as features or targets. String columns (composition strings, file paths, structure names) are automatically excluded and logged for your reference.

3.5 Train/Test Split

Configure how your data is divided between training and testing:

- Test Size: Percentage of data reserved for testing (default: 20%)
- Random State: Seed for reproducible splits. Changing this value now produces genuinely different results (v1.4.0 fix)
- External Validation: Load a separate validation dataset for independent evaluation

Note: In v1.4.0, the random state functionality was completely fixed. The train/test split uses the seed from Custom Parameters, KFold CV uses the model's `random_state`, and NN CV uses `TrainingConfig.random_seed`. Models without `random_state` (SVR, KNN) remain unaffected.

4. Training Tab

The Training tab is where you train multiple machine learning algorithms simultaneously. AutoAIM Studio supports seven regression algorithms with cross-validation and comprehensive result tracking.

4.1 Algorithm Selection

Select one or more algorithms to train:

Algorithm	Type	Key Characteristics
Random Forest	Ensemble (bagging)	Robust, handles non-linear relationships well
Gradient Boosting	Ensemble (boosting)	Sequential error correction, high accuracy
XGBoost	Gradient boosting	Optimized implementation, regularization
LightGBM	Gradient boosting	Leaf-wise growth, fast training
CatBoost	Gradient boosting	Handles categorical features natively
Support Vector Regression	Kernel method	Effective in high-dimensional spaces
k-Nearest Neighbors	Instance-based	Simple, interpretable

Pro Tip: For most materials science datasets, we recommend starting with Random Forest, XGBoost, and LightGBM. These tree-based methods handle the non-linear relationships common in materials data.

4.2 Cross-Validation

Cross-validation (CV) provides robust performance estimates by training on multiple data splits. Configure the number of folds (1-10) in the Training tab. When CV folds > 1, AutoAIM performs K-fold cross-validation with fresh model instances per fold.

- CV = 1: Single train/test split (fastest)
- CV = 5: Standard 5-fold cross-validation (recommended)
- CV = 10: 10-fold cross-validation (most robust)

The Model Details dialog shows a table of per-fold CV scores for all model types in both Training and Results tabs.

4.3 Train with Custom Parameters

New in v1.3.0 and enhanced in v1.4.0, the "Train with Custom Parameters" feature allows you to manually configure any regressor's hyperparameters and train directly, bypassing Bayesian optimization. This is ideal when you know the parameters you want or need to test specific configurations.

To use Custom Parameters:

7. Select an algorithm from the dropdown
8. Click "Train with Custom Parameters"
9. Configure parameters in the dialog (see table below)
10. Set Random State and n_jobs for reproducibility
11. Click "Train"

Algorithm	Parameters
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features
Gradient Boosting	n_estimators, max_depth, learning_rate, subsample, min_samples_split, min_samples_leaf
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree, reg_alpha, reg_lambda, min_child_weight, gamma
LightGBM	n_estimators, max_depth, learning_rate, num_leaves, subsample, colsample_bytree, reg_alpha, reg_lambda, min_child_samples
CatBoost	iterations, depth, learning_rate, l2_leaf_reg
SVR	C, epsilon, gamma (scale/auto/numeric), kernel (linear/poly/rbf/sigmoid)
Ridge	alpha

Important: In v1.4.0, the Custom Parameters dialog supports 10-12 decimal places (was 6-8) for fine-grained sensitivity testing. Additionally, all manually-configurable parameters are now included in Bayesian optimization, ensuring complete coverage.

Each parameter has an educational tooltip explaining what it does and typical values. Hover over any spinbox, combo, or checkbox to see the tooltip. AutoAIM Studio includes 53+ educational tooltips across all configuration tabs.

Pro Tip: The High-Precision Input feature (v1.4.0) allows you to enter values like 0.0000001 for parameters like min_delta, enabling fine sensitivity testing that was not possible with the previous 6-8 decimal limit.

4.4 Understanding Results

After training completes, the Results tab shows:

- Model name and algorithm type
- R2, RMSE, MAE, and MAPE metrics
- CV score (mean cross-validation score)
- Per-fold CV score breakdown (click "Details")
- Training time

Note: In v1.3.0+, sequential trained models (e.g., XGBoost_2, Neural Network_2) now preserve all results instead of overwriting. The `_get_unique_model_name()` function ensures proper sequential naming for all model types.

5. Optimization Tab

The Optimization tab provides Bayesian hyperparameter optimization using Optuna. This powerful feature automatically searches for the best combination of hyperparameters for your selected algorithm, often achieving significantly better performance than default settings.

5.1 Bayesian Optimization

To run optimization:

12. Select the algorithm to optimize from the dropdown
13. Configure the number of Optuna trials (more trials = better results, but slower)
14. Optionally set custom parameter ranges (click "Set Ranges")
15. Click "Optimize"

During optimization, AutoAIM displays a progress bar, current trial number, and best score found so far. After completion, the results show:

- Ranked list of all trials with scores and parameters
- Best Trial Details panel with per-fold CV scores
- Convergence plot showing optimization progress

Materials Science Context: Bayesian optimization builds a probabilistic model of the objective function (model performance) and uses it to select the most promising hyperparameters to evaluate next. This is far more efficient than grid search or random search, especially for high-dimensional spaces.

5.2 Select Parameters to Optimize

New in v1.4.0, the "Select Parameters to Optimize" feature is the most impactful addition for CPU users. You can now choose which hyperparameters Optuna will optimize, with a "Recommended" preset that selects only high-impact parameters for faster optimization.

The dialog provides:

- Checkbox for every parameter of every model
- "Select All" - optimize everything (default behavior)
- "Select Recommended" - only high-impact parameters (faster on CPU)
- "Clear All" - start from scratch
- Educational tooltips on each parameter explaining its impact level

Mode	Description	Best For
Select All	Every parameter is optimized	Maximum accuracy, GPU users
Select Recommended	Only high-impact parameters	Faster optimization, CPU users
Clear All + Custom	Manually select specific parameters	Targeted optimization, expert users

Pro Tip: For CPU-based optimization, use 'Select Recommended' mode. This typically reduces the search space by 40-60% while maintaining 95%+ of the potential performance improvement.

Important: The Select Parameters to Optimize feature is available for both regressor optimization (Optimization Tab) and NN architecture optimization (NN Tab). In v1.4.0, LightGBM's min_child_samples was added, bringing it to 9 tunable parameters matching XGBoost's coverage.

Full parameter coverage by algorithm (v1.4.0):

Algorithm	# Params	Optimized Parameters
Random Forest	5	n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features
Gradient Boosting	6	n_estimators, max_depth, learning_rate, subsample, min_samples_split, min_samples_leaf
XGBoost	9	n_estimators, max_depth, learning_rate, subsample, colsample_bytree, reg_alpha, reg_lambda, min_child_weight, gamma
LightGBM	9	n_estimators, max_depth, learning_rate, num_leaves, subsample, colsample_bytree, reg_alpha, reg_lambda, min_child_samples

CatBoost	4	iterations, depth, learning_rate, l2_leaf_reg
SVR	4	C, epsilon, gamma, kernel (categorical)
Ridge	1	alpha

5.3 Optimization Results

The optimization results tab (v1.3.0+) provides a clean 4-column layout showing Rank, Score, Parameters, and Trial#. CV fold scores are displayed in the "Best Trial Details" text panel in the same tabular format as the Training Tab.

After optimization completes:

- Review the best parameters found
- Examine per-fold CV scores in Best Trial Details
- Check the convergence plot for optimization behavior
- Use "Train with Best Parameters" for instant deployment

5.4 Train with Best Parameters

After optimization, click "Train with Best Parameters" to immediately train a model using the best hyperparameters found. This creates a fully trained model that appears in the Results tab with all standard metrics and export options.

Note: In v1.3.0+, the TrainBestThread overwrite bug was fixed. Sequential optimized models now use proper sequential naming (`_get_unique_model_name`) instead of deleting previous results.

6. Neural Network Tab

The Neural Network tab provides a visual interface for designing and training custom PyTorch architectures. You can add layers, configure activations, set training parameters, and optimize the entire architecture using Bayesian optimization.

6.1 Architecture Design

Build your network by adding and configuring layers:

- Dense (Linear): Fully connected layers - the building blocks of most networks
- Dropout: Regularization layer that prevents overfitting by randomly disabling neurons during training
- BatchNorm: Normalizes layer inputs to stabilize and accelerate training
- Activation: ReLU, LeakyReLU, GELU, Sigmoid, Tanh, or ELU

Pro Tip: For materials property prediction, we recommend starting with 2-3 hidden layers of 64-128 units each, with ReLU activation and 0.2-0.3 dropout between layers.

6.2 Training Configuration

Configure training parameters in the right panel:

Parameter	Description	Range
Epochs	Number of training iterations	10-10,000
Learning Rate	Step size for gradient descent	1e-5 to 1.0
Batch Size	Samples per gradient update	8-512
Optimizer	Optimization algorithm	Adam, SGD, AdamW, RMSprop
LR Scheduler	Learning rate adjustment strategy	reduce_on_plateau, cosine_annealing, step_lr, exponential, none
Min Delta	Early stopping minimum improvement	1e-6 to 0.1 (default: 1e-4)
Gradient Clip	Maximum gradient norm	0 to 10 (default: 1.0, 0 = disabled)
CV Folds	Cross-validation folds	1-10
Random Seed	Reproducibility seed	Any integer

Note: The LR Scheduler, Min Delta, and Gradient Clip parameters were added in v1.3.0. Gradient clipping is particularly useful for preventing exploding gradients in deep networks.

6.3 NN Optimization

Bayesian optimization for neural networks searches across architecture and training hyperparameters simultaneously. In v1.4.0, you can use Select Parameters to Optimize to focus the search on specific parameters.

Optimizable parameters include:

- Layers: Number of hidden layers (1-5)
- Units: Neurons per layer (16-256)
- Dropout: Dropout rate (0.0-0.5)
- Activation: ReLU, LeakyReLU, GELU, ELU (v1.3.0+)
- Learning Rate (log scale)
- Batch Size: 16, 32, 64, 128, 256 (categorical)
- Optimizer: Adam, SGD, AdamW (categorical)
- Gradient Clip Val: 0.0-5.0 (v1.3.0+)

After optimization, click "Apply Best to Architecture" to rebuild the network with optimized hyperparameters. The model re-trains automatically with the new configuration.

Note: In v1.3.0+, the NN Post-Optimization Status feature switches the progress bar to animated indeterminate mode after the last Optuna trial while CV compilation is in progress, preventing the 'frozen UI' appearance.

6.4 Training Results

New in v1.4.0, the Training Results table in the NN Tab now shows epoch-by-epoch data with Train Loss, Val Loss, Train Metric, Val Metric, and Time for each epoch. This provides detailed insight into the training dynamics and helps identify overfitting or convergence issues.

Pro Tip: The NN Tab was redesigned in v1.4.0 with a compact layout: Actions + Presets merged into a single toolbar, all 4 right panels have limited height (no more oversized empty areas), and more horizontal space for the configuration panel.

7. Ensemble Tab

The Ensemble tab allows you to combine multiple trained models for improved predictions. AutoAIM supports weighted average and stacking strategies, with optional Bayesian optimization of member weights.

7.1 Creating Ensembles

To create an ensemble:

16. Train two or more individual models in the Training tab
17. Switch to the Ensemble tab
18. Select the models to include (check the boxes)
19. Choose strategy: Weighted Average or Stacking
20. Configure CV folds for ensemble evaluation
21. Click "Train Ensemble"

Important: In v1.4.0, two critical ensemble bugs were fixed: (1) Ensemble results not appearing in the Results Tab - caused by calling `get_ensemble_trainer()` which did not exist, now changed to direct attribute access. (2) Ensemble results being duplicated - removed code that saved ensembles to both `ensemble_trainer.results` and `trainer.results`.

7.2 Manual Ensemble Weights

New in v1.3.0, the Manual Weights panel allows you to set custom weights for each model in a weighted-average ensemble. Configure per-model weight spinboxes and click "Train with Manual Weights" to train the ensemble using your defined weights. If no manual weights are set, the system falls back to equal weights.

Pro Tip: Manual weights are useful when you have domain knowledge about which models should contribute more to the final prediction. For example, you might weight XGBoost higher if you know it performs well on your specific type of materials data.

7.3 Weight Optimization

Click "Optimize Weights" to run Bayesian optimization on ensemble member contributions. Optuna searches for the optimal weight combination that maximizes ensemble performance. This often produces better results than equal or manually-set weights.

8. Results Tab

The Results tab provides a comprehensive overview of all trained models, including regressors, ensembles, and neural networks. The redesigned interface (v1.3.0+) deduplicates results by name before rendering the comparison table.

For each model, you can view:

- Model name and algorithm type
- R2, RMSE, MAE, MAPE metrics
- CV score with per-fold breakdown (click "Details")
- Training time and status

Available actions:

- Delete: Remove a model from results (v1.4.0: now searches `training_tab.training_results` too)
- Export: Save model as `.joblib` or `.pt` file
- Details: View per-fold CV scores and detailed metrics
- Create Bundle: Package model for standalone deployment

Note: In v1.4.0, the manual model deletion bug was fixed. Delete now searches in `training_tab.training_results` as well as `trainer.results`, ensuring models can always be removed regardless of how they were created.

9. Explainability Tab

The Explainability tab provides model interpretation tools to understand which features drive your predictions. This is crucial for scientific validity and domain insights.

Available analyses:

- SHAP Summary Plot: Global feature importance based on SHAP values
- Permutation Importance: Feature importance measured by shuffling values
- Partial Dependence: How predictions change as a feature varies

- Individual Predictions: SHAP force plots for single predictions

Materials Science Context: SHAP (SHapley Additive exPlanations) values come from cooperative game theory and provide a theoretically sound way to attribute predictions to features. A feature with high SHAP importance consistently influences model predictions across the dataset.

Pro Tip: Use SHAP summary for global understanding, permutation importance to verify robustness, and partial dependence to visualize feature-prediction relationships.

10. Predict Tab

The Predict tab enables standalone prediction using trained models without the full application. You can load a Model Bundle or a trained model file and make predictions on new data.

For crystal structure prediction (v1.2.0+):

- Load CIF, POSCAR, or XYZ files directly
- Structures are automatically featurized using the same pipeline as training
- No external CSV metadata required
- Prediction exports preserve structure_name as the first column

Note: The Inference tab detects models trained with the old one-hot encoding bug (400+ features) and warns you to retrain. This ensures prediction compatibility.

11. Model Bundles

Model Bundles are self-contained packages that include everything needed for standalone prediction. Each bundle contains the trained model, preprocessing pipeline, feature list, and a manifest.json with complete metadata.

To create a Model Bundle:

22. Train a model in the Training or Optimization tab
23. In the Results tab, click "Create Bundle"
24. Choose output location
25. The .zip bundle is created with all necessary files

Bundle contents:

- model.joblib or model.pt: The trained model
- manifest.json: Metadata (algorithm, parameters, feature list, metrics)
- preprocessor.joblib: Data preprocessing pipeline
- prediction_script.py: Standalone prediction script

Important: In v1.3.0+, the GPU Model Bundle Save bug was fixed. TorchScript conversion now moves the model to CPU before tracing. If TorchScript fails, the full nn.Module is saved as fallback. The inference engine auto-detects the format (TorchScript vs full model) on load.

12. Troubleshooting

This section covers common issues and their solutions. For additional support, contact us at veradelagarza@quimica.unam.mx

Issue	Solution
"Word found unreadable content" when opening docs	Use the latest python-docx generated files (v1.4.0+). Earlier C# builds had OPC packaging issues.
Ensemble results not appearing in Results Tab	Fixed in v1.4.0. Update to the latest version.
Manual models not deletable	Fixed in v1.4.0. Delete now searches training_tab.training_results too.
SVR Gamma Crash: 'could not convert string to float'	Fixed in v1.3.0. Gamma is now a combo box accepting 'scale', 'auto', or numeric values.
CatBoost 'random_seed, random_state' conflict	Fixed in v1.3.0. Custom Parameters detects CatBoost and uses random_seed.
PyInstaller 'periodic_table.json.gz' missing	Fixed in v1.2.0. Runtime hook auto-configures PMG_HOME.
Feature explosion (771+ features)	Fixed in v1.2.0. Only numeric columns used as features; string columns excluded.
Large datasets causing RAM issues	Reduce dataset size or increase system RAM. Close other applications.
GPU not detected for NN training	Run from source with CUDA-enabled PyTorch. PyInstaller executable is CPU-only.
Optimization seems slow on CPU	Use 'Select Recommended' mode in v1.4.0 to reduce search space.
Random state changes not affecting results	Fixed in v1.4.0. All model types now properly use configurable random seeds.

12.1 Error Reporting

In v1.4.0, all training and optimization threads now include full traceback in error messages. This makes debugging significantly easier. Additionally, the `_safe_get_parent_attr()` helper replaces approximately 50 direct `self.parent.*` accesses across all GUI tabs, preventing `AttributeError` crashes from race conditions.

12.2 Logging

In v1.4.0, all `print()` calls were replaced with proper logging (`logger.info/debug/warning/error`) across the GUI. Logs are written to the application directory and can help diagnose issues.

13. Version History

Version	Date	Codename	Key Features
1.4.0	2026-06-27	Full Control	Select Parameters to Optimize, Random State fix, High-Precision Input, LightGBM <code>min_child_samples</code> , NN Tab redesign, Training Results, ensemble fixes, safe parent access, unified logging
1.3.0	2026-06-24	Precision	Custom Parameters (7 regressors), Manual Ensemble Weights, 53 tooltips, CV fold breakdown, LR Scheduler/Min Delta/Gradient Clip, NN Post-Optimization status, tab reorder, GPU bundle fix
1.2.0	2026-06-14	Crystal	Crystal Structure Support (CIF/POSCAR/XYZ), ~154 descriptors, SVR/KNN optimization fix, PyInstaller fix, feature explosion fix
1.1.0	2026-05-18	Foundation	NN optimization, CV for all models, Ensemble Weight Optimization, Apply Best Parameters
1.0.0	2026-04-08	Genesis	Initial stable release, AutoML pipeline, compositional featurization, Model Bundles

For the complete changelog with all bug fixes and technical details, see `CHANGELOG.md` included with your AutoAIM Studio distribution.

14. Citation

If you use AutoAIM Studio in your research, please cite:

```
@software{autoaim_studio_2026,  
  title = {AutoAIM Studio: Auto Artificial Intelligence for Materials},  
  version = {1.4.0},  
  year = {2026},  
  doi = {10.5281/zenodo.20947495},  
  url = {https://github.com/cesargabrielvera1-ux/AutoAIM-Studio}  
}
```

Contact: veradelagarza@quimica.unam.mx

DOI: 10.5281/zenodo.20947495

Thank you for using AutoAIM Studio! We hope this tool accelerates your materials discovery and research. For questions, suggestions, or bug reports, please contact us at the email above.